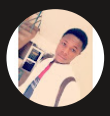
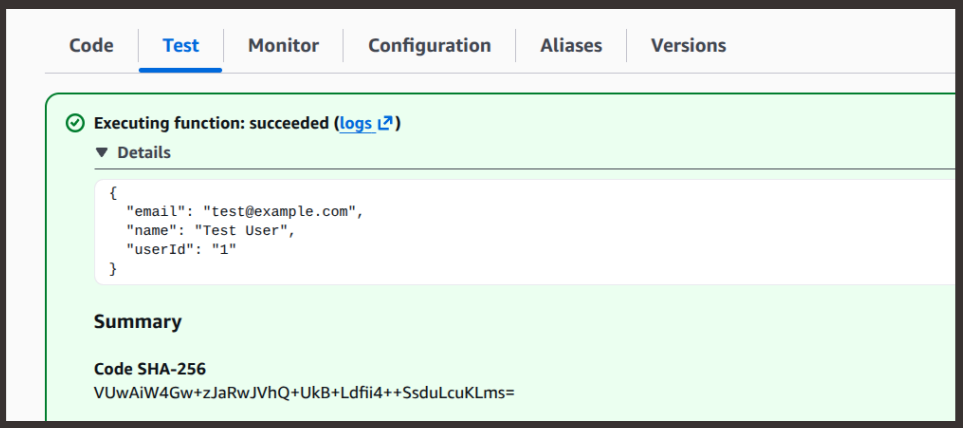
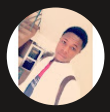
nextwork.org

Fetch Data with AWS Lambda



Ahmed Tetteh





Ahmed Tetteh
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate a data logic of my Three tier web project, by creating a creating a serverless function that restrieves user data from a database.

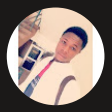
Tools and concepts

Services I used were Lambda and DynamoDB. Key concepts I learnt include Lambda functions how to set up a DynamoDB database. creating, configuring and testing a Lambda function.

Project reflection

This project took me approximately 1 hour 20 mins. The most challenging part was learning how integrate Lambda with other services using an Execution role and the right permissions attached. It was most rewarding to tighten up the security around my Lambda function, but still seein it perform exactly as intented.

I chose to do this project today because it showed me how serverless architectures work in the real-world.

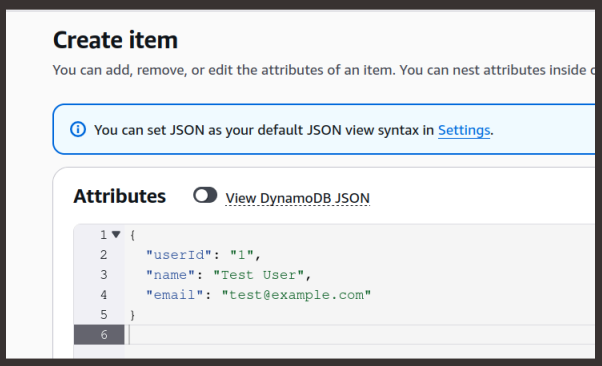


Ahmed Tetteh
NextWork Student

Project Setup

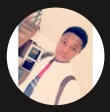
To set up my project, I created a database table and a simple primary key. The simple primary key consists of the partition key, which is a hash value used to retrieve items from the table. This is how DynamoDB speads the data across multiple instances to perform efficient quering.

In my DynamoDB table, I added a user data with some few more attributes, such as a name and an email. DynamoDB is schemaless, which means you don't need to know all the attributes before hand to create your data or populate the table. You can add different attributes and different items can belong to different sets of attributes.



AWS Lambda

AWS Lambda is a service that runs code without the need for developers without provisioning or managing servers . I'm using Lambda in this project to trigger a function(code) that retrieves data from my DynamoDB table.



Ahmed Tetteh
NextWork Student

AWS Lambda Function

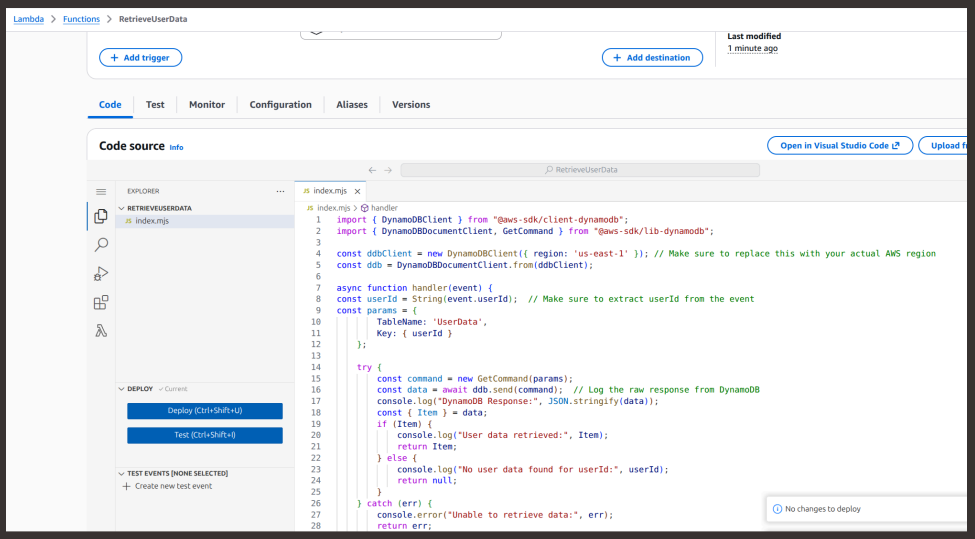
My Lambda function has an execution role, which is simply the role it needs to perform some basic fuctions. By default, the role grants Lamdda the permission to wite logs to CloudWatch.

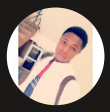
My Lambda function will interact with DynamoDB. It takes in a UserId, checks the DynamoDB table for a corresponding ID, and returns the data from the table. Else, it returns an error messge or data not found.

The code uses AWS SDK, which is a library of pre-built code that allows developers to interact with AWS services. My code uses SDK to interact with DynamoDB.

Ahmed Tetteh
NextWork Student

nextwork.org





Ahmed Tetteh
NextWork Student

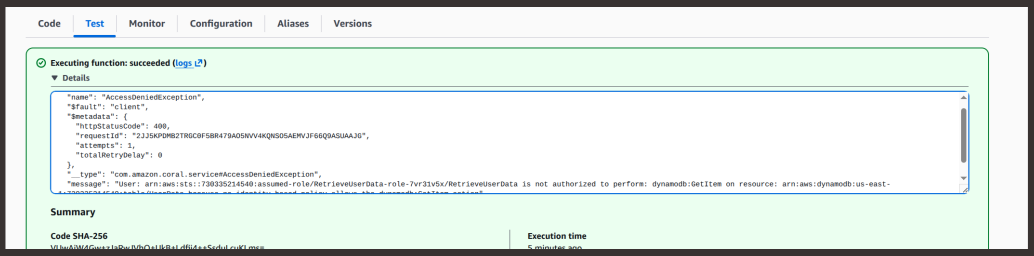
Function Testing

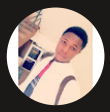
To test whether my Lambda function works, I ran a sample test event with a UserId. The test is written in JSON format, making it very easy for our Lambda function to interprete, regardless of the runtime we choose. If the test is successful, I'd see "Executing function: succeeded" message.

The test displayed a 'success' because the function itself run without any errors in the code, but the function's response was actually an error message because my Lambda Function's execution role doesn't contain the permissions to access the DynamoDB table, hence, it was blocked by DynamoDB.

Ahmed Tetteh
NextWork Student

nextwork.org





Ahmed Tetteh
NextWork Student

Function Permissions

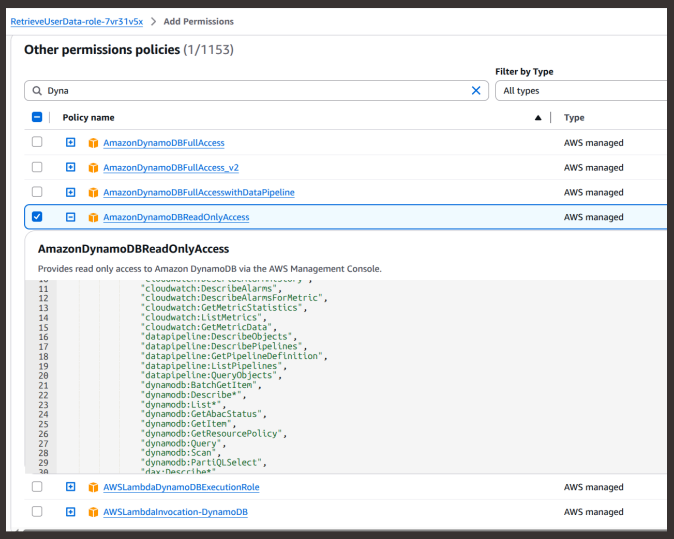
To resolve the AccessDenied error, I reviewed the error log to determine the exact type of permissions my Lambda Function will need to interact with DynamoDB.

There were four DynamoDB permission policies I could choose from, but I didn't pick "AWSLambdaDynamoDBExecutionRole", "AWSLambdaInvocation-DynamoDB" and "AmazonDynamoDBFullAccess" because they didn't correspond to the right permissions my Lambda Function needed to do its job, i.e, "dynamodb:GetItem".

I also didn't pick "AmazonDynamoDBFullAccess" because full access does not comply with the least priviledge rule in AWS for a resource or entity, making it less secure if compromised. AmazonDynamoDBReadOnlyAccess was the right choice because it contained the exact permissions my Lambda Function needed to its job, without being compromising my DynamoDB table. Also, my Lambda Function just needs to read and retrieve items from the table, not write.

Ahmed Tetteh
NextWork Student

nextwork.org

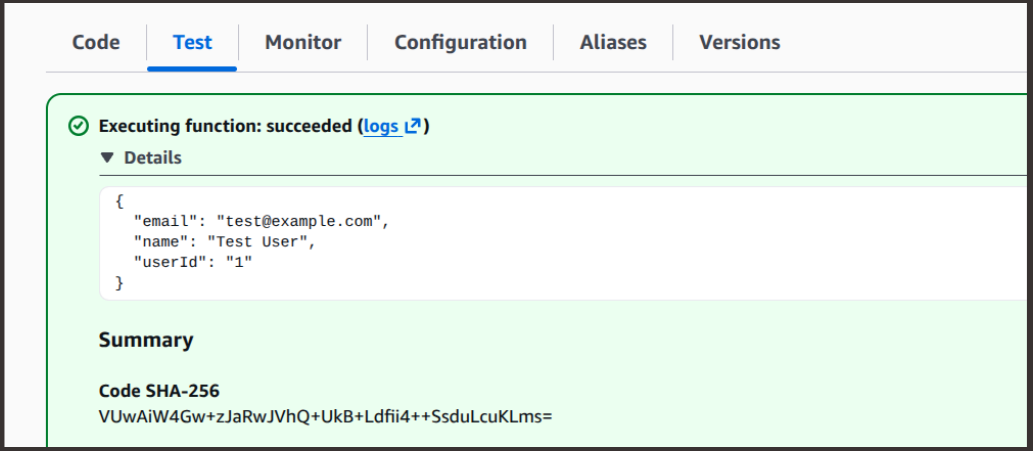


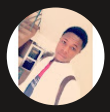
Ahmed Tetteh
NextWork Student

Final Testing and Reflection

To validate my new permission settings, I re-run my test event again. The results were succesful, as expected because my Lambda Function now had the right permissions to retrieve items from the table.

Web apps are a popular use case of using Lambda and DynamoDB. For example, I could use Lambda to retrieve cart items from a DynamoDB table on a product site or even get product information.





Ahmed Tetteh
NextWork Student

Enahancing Security

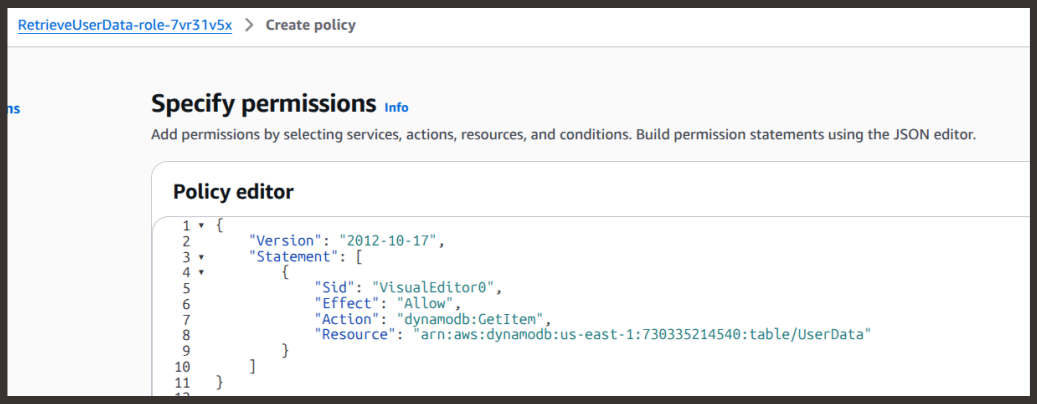
For my project extension, I challenged myself to tighten the security of my DynamoDB table by creating an Inline policy for my Lambda Function.

To create the permission policy, I used an Inline policy because it allows me to provides even more granular control over a resource, eliminating all other unecessary permissions the resource will not use - making it more secure.

I verified that my Lambda function still works by re-running a test event to validate if the Function still works as expected.

Ahmed Tetteh
NextWork Student

nextwork.org



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects